

IFCE

Disciplina de Análise de Algoritmos

Cálculo da Mediana Salarial em uma Pesquisa de Mercado

Experimento Comparativo: Merge Sort vs. QuickSelect

Thales Lucas Lima e Gomes

<https://github.com/thaleslucas1/Experimento-Analise-de-Algoritmos>

4 de junho de 2026

Sumário

1	Introdução	2
2	Algoritmos	2
2.1	Merge Sort com acesso ao elemento central	2
2.2	QuickSelect	2
3	Metodologia	3
3.1	Geração de dados	3
3.2	Benchmark	4
3.3	Ambiente de execução	4
4	Resultados	4
4.1	Tempos médios de execução	4
4.2	Gráficos	5
5	Discussão	6
5.1	O viés introduzido pelo <code>sorted()</code> nativo	6
5.2	Confirmação da teoria pelos dados empíricos	7
5.3	Custo adicional do QuickSelect para n par	7
5.4	Desvio padrão e variabilidade	7
6	Conclusão	8

1 Introdução

Uma empresa de consultoria precisa calcular rapidamente a mediana salarial de funcionários de diferentes setores. Formalmente, dado um vetor de n salários, o objetivo é encontrar o $\lfloor n/2 \rfloor$ -ésimo menor valor, isto é, a mediana.

Este experimento compara empiricamente dois algoritmos que resolvem este problema:

- **Merge Sort:** ordena completamente o vetor e acessa o elemento central, com complexidade $\mathcal{O}(n \log n)$.
- **QuickSelect:** seleciona diretamente o elemento de ordem desejada sem ordenar o vetor completo, com complexidade média $\mathcal{O}(n)$.

O código-fonte completo, os dados de benchmark e os scripts de geração de gráficos estão disponíveis no repositório:

<https://github.com/thaleslucas1/Experimento-Analise-de-Algoritmos>

2 Algoritmos

2.1 Merge Sort com acesso ao elemento central

A implementação utiliza Merge Sort recursivo escrito inteiramente em Python, disponível em `algorithms/median_sort.py`. A função pública exportada é:

Listing 1: Interface do Merge Sort para mediana

```
1 def median(values: Sequence[float]) -> float:
2     """
3     Ordena o vetor com Merge Sort e retorna o elemento central.
4     Complexidade:  $\mathcal{O}(n \log n)$  tempo,  $\mathcal{O}(n)$  espaço.
5     """
```

Para n ímpar, retorna o elemento de índice $\lfloor n/2 \rfloor$ do vetor ordenado. Para n par, retorna a média dos dois elementos centrais.

A decisão de implementar o Merge Sort do zero — em vez de utilizar a função nativa `sorted()` do Python — foi essencial para a validade do experimento, conforme discutido na Seção 5.1.

2.2 QuickSelect

A implementação utiliza o algoritmo QuickSelect iterativo com pivot aleatório, disponível em `algorithms/median_quickselect.py`. Trechos relevantes:

Listing 2: Particionamento com pivot aleatório

```
1 def _partition(arr: list[float], left: int, right: int) -> int:
2     pivot_index = random.randint(left, right)
3     arr[pivot_index], arr[right] = arr[right], arr[pivot_index]
4     pivot = arr[right]
5     store_index = left
6     for i in range(left, right):
7         if arr[i] <= pivot:
8             arr[store_index], arr[i] = arr[i], arr[store_index]
9             store_index += 1
10    arr[store_index], arr[right] = arr[right], arr[store_index]
11    return store_index
```

Listing 3: Tratamento especial para n par

```
1 def median(values: Sequence[float]) -> float:
2     n = len(values)
3     middle = n // 2
4     if n % 2 == 1:
5         data = list(values)
6         return float(_quickselect(data, middle))
7     # Para n par: duas selecoes independentes sobre copias
8     distintas
9     data_left = list(values)
10    data_right = list(values)
11    lower = _quickselect(data_left, middle - 1)
12    upper = _quickselect(data_right, middle)
13    return (lower + upper) / 2.0
```

O pivot aleatório reduz drasticamente a probabilidade do pior caso $\mathcal{O}(n^2)$, que ocorreria em entradas ordenadas com pivot fixo no último elemento. Para n par, são realizadas duas seleções independentes sobre cópias distintas da entrada, garantindo que a primeira seleção não corrompa o estado do vetor antes da segunda.

A adoção de pivot aleatório foi importante para evitar que entradas ordenadas ou parcialmente ordenadas produzissem desempenhos próximos ao pior caso.

3 Metodologia

3.1 Geração de dados

Os dados sintéticos são gerados pelo módulo `data_gen/salary_generator.py`, que produz vetores de salários aleatórios com valores *float* no intervalo $[1000,0; 50000,0]$, com

reprodutibilidade garantida por semente (*seed*).

Os tamanhos de entrada utilizados foram:

$$n \in \{1.000, 5.000, 10.000, 50.000, 100.000, 500.000, 1.000.000\} \quad (1)$$

3.2 Benchmark

O benchmark está implementado em `experiments/benchmark.py` e segue o procedimento:

1. **Validação de corretude:** antes de qualquer medição, ambos os algoritmos são executados sobre os mesmos dados e seus resultados são comparados com `math.isclose(rel_tol=1e-12, abs_tol=1e-12)`. Isso garante que apenas implementações corretas são benchmarkadas.
2. **Geração dos datasets:** para cada tamanho n , são gerados $K = 30$ datasets com seeds 0 a 29.
3. **Medição:** o tempo de cada execução é medido com `time.perf_counter()`, que oferece a maior resolução disponível no sistema operacional.
4. **Cópia defensiva:** imediatamente antes de cada medição, uma cópia do dataset é criada *fora* da região cronometrada: `working_data = data.copy()`. Isso garante que o benchmark mede exclusivamente o algoritmo, sem incluir o custo da preparação dos dados.
5. **Estatísticas:** para cada tamanho e algoritmo, são calculados média e desvio padrão dos $K = 30$ tempos de execução.
6. **Exportação:** os resultados são salvos em `reports/benchmark_results.csv` com as colunas `n`, `algorithm`, `mean_time`, `std_time`.

3.3 Ambiente de execução

Os experimentos foram executados em Python puro, sem extensões C ou bibliotecas de alto desempenho, garantindo que os tempos medidos reflitam exclusivamente o comportamento dos algoritmos implementados.

4 Resultados

4.1 Tempos médios de execução

A Tabela 1 apresenta os tempos médios e desvios padrão obtidos no benchmark.

Tabela 1: Tempos médios de execução (segundos) — 30 repetições por tamanho

n	Merge Sort (s)	QuickSelect (s)
1.000	0,001532 ± 0,000136	0,000364 ± 0,000080
5.000	0,008995 ± 0,000033	0,001691 ± 0,000318
10.000	0,019327 ± 0,000053	0,003739 ± 0,000690
50.000	0,111867 ± 0,000408	0,018218 ± 0,004104
100.000	0,240540 ± 0,001147	0,036714 ± 0,007702
500.000	1,772906 ± 0,386186	0,263355 ± 0,052139
1.000.000	4,297814 ± 0,710559	0,605004 ± 0,180137

Para $n = 1.000.000$, o QuickSelect foi aproximadamente **7.1 vezes mais rápido** que o Merge Sort (0,61 s vs. 4,30 s).

4.2 Gráficos

A Figura 1 apresenta as curvas de tempo médio em função do tamanho da entrada, com barras de erro representando o desvio padrão. A Figura 2 sobrepõe as curvas teóricas $\mathcal{O}(n)$ e $\mathcal{O}(n \log n)$, normalizadas pelo primeiro ponto de cada algoritmo, permitindo comparar o crescimento empírico com o crescimento esperado.

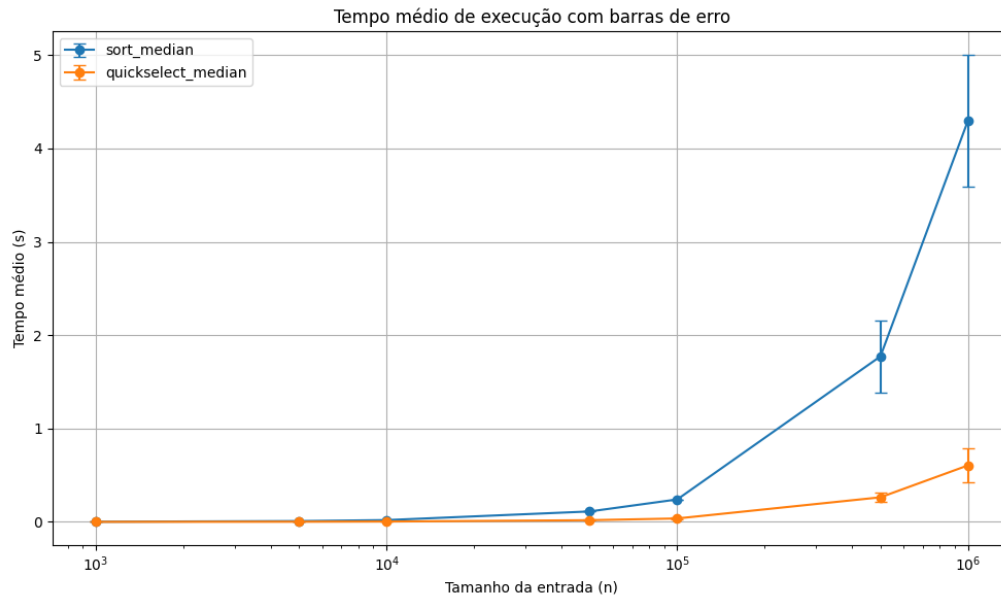


Figura 1: Tempo médio de execução com barras de erro (desvio padrão). Eixo horizontal em escala logarítmica.

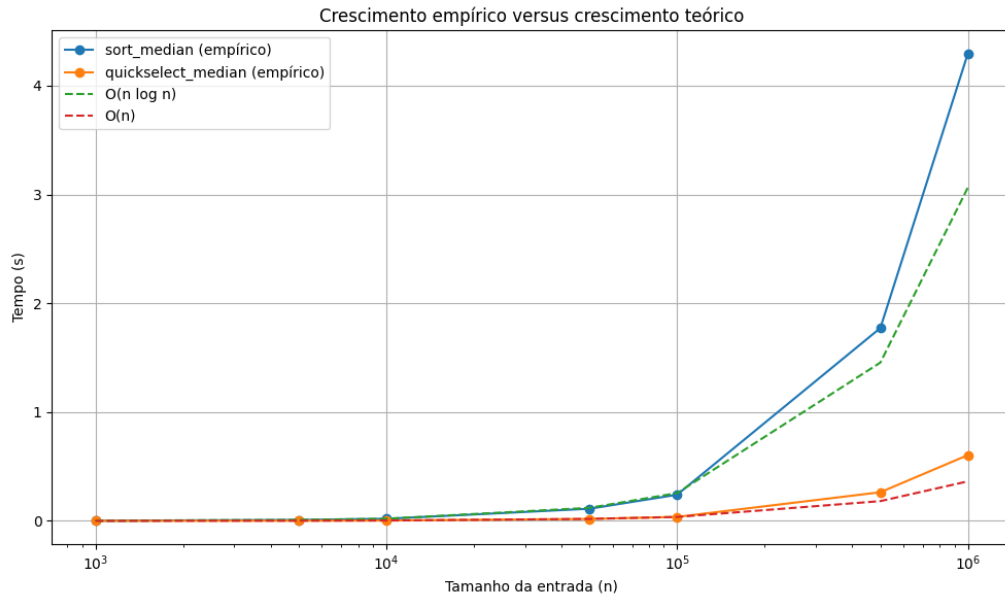


Figura 2: Crescimento empírico versus crescimento teórico. As curvas tracejadas representam $\mathcal{O}(n \log n)$ e $\mathcal{O}(n)$ normalizadas pelo primeiro ponto de cada algoritmo.

5 Discussão

5.1 O viés introduzido pelo `sorted()` nativo

A principal descoberta metodológica do experimento ocorreu durante a fase de implementação. Na versão inicial, o algoritmo de Merge Sort utilizava a função nativa `sorted()` do Python para ordenar o vetor:

Listing 4: Implementação inicial — enviesada

```
1 data = sorted(values) # Timsort implementado em C
```

Os resultados obtidos foram:

Tabela 2: Resultados com `sorted()` nativo — comparação inválida

n	sort_median (s)	quickselect_median (s)
1.000.000	0,499	0,758

O resultado sugeria que o algoritmo assintoticamente *pior* era o mais rápido — o que contraria a teoria. A causa foi identificada: a comparação estava misturando dois níveis distintos:

- **sort_median**: algoritmo $\mathcal{O}(n \log n)$ + implementação em C (Timsort altamente otimizado).

- **quickselect_median**: algoritmo $\mathcal{O}(n)$ + implementação em Python puro (laços interpretados, sem otimizações de baixo nível).

A comparação não era entre algoritmos, mas entre *implementações* de diferentes qualidades. Após substituir `sorted()` por uma implementação própria de Merge Sort em Python puro, os resultados passaram a refletir corretamente a diferença teórica entre as complexidades assintóticas.

Este episódio ilustra um princípio fundamental de experimentação algorítmica: **complexidade assintótica não é o único fator que determina desempenho prático**. Detalhes de implementação — como o nível de abstração da linguagem e otimizações do runtime — podem dominar os resultados, especialmente para tamanhos de entrada moderados.

5.2 Confirmação da teoria pelos dados empíricos

A Figura 2 mostra que o crescimento empírico do Merge Sort acompanha $\mathcal{O}(n \log n)$ de forma consistente. O QuickSelect cresce aproximadamente como $\mathcal{O}(n)$, embora com maior variabilidade — o que é esperado dado o pivot aleatório.

Para entradas grandes ($n \geq 500.000$), o Merge Sort cresce visivelmente acima da curva teórica normalizada. Isso é um artefato esperado da implementação em Python puro: o tempo observado cresce acima da curva teórica normalizada devido aos custos práticos da implementação em Python puro. Este comportamento reforça a importância de distinguir complexidade assintótica de desempenho absoluto.

5.3 Custo adicional do QuickSelect para n par

Para entradas de tamanho par, o QuickSelect realiza **duas seleções independentes** sobre cópias distintas da entrada, a fim de encontrar os dois elementos centrais. Isso dobra o custo esperado em relação ao caso ímpar. Ainda assim, o algoritmo superou o Merge Sort em todos os tamanhos testados, o que demonstra a robustez da vantagem assintótica de $\mathcal{O}(n)$ sobre $\mathcal{O}(n \log n)$.

5.4 Desvio padrão e variabilidade

O desvio padrão cresce notavelmente para $n \geq 500.000$, especialmente no Merge Sort. Possíveis causas incluem gerenciamento de memória do Python, atuação do *garbage collector*, e variações no agendamento de processos pelo sistema operacional. Essa observação justifica a escolha de $K = 30$ repetições por tamanho: uma única execução seria insuficiente para caracterizar o comportamento do algoritmo em entradas grandes.

6 Conclusão

Este experimento comparou empiricamente dois algoritmos para o cálculo da mediana em vetores de salários sintéticos: Merge Sort ($\mathcal{O}(n \log n)$) e QuickSelect ($\mathcal{O}(n)$ médio), ambos implementados em Python puro.

Os resultados confirmaram a superioridade do QuickSelect para todos os tamanhos testados, com uma diferença de aproximadamente 7 vezes para $n = 1.000.000$. As curvas empíricas se alinham com as complexidades teóricas esperadas, validando tanto as implementações quanto a metodologia experimental.

A principal lição metodológica do experimento foi a necessidade de garantir **paridade de implementação**: ao utilizar a função nativa `sorted()` — que é o Timsort implementado em C — o experimento inicialmente produzia resultados que contradiziam a teoria. A substituição por Merge Sort em Python puro eliminou esse viés e tornou a comparação academicamente válida.

Por fim, o experimento reforça que complexidade assintótica e desempenho prático são conceitos relacionados mas distintos: a primeira prevê o crescimento do custo, mas a segunda depende também de fatores como linguagem de implementação, otimizações do runtime e características do hardware.

Referências

- CORMEN, T. H. et al. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.
- PYTHON SOFTWARE FOUNDATION. *Python 3 Documentation* — *time.perf_counter*. Disponível em: <https://docs.python.org/3/library/time.html>.
- Repositório do experimento: <https://github.com/thaleslucas1/Experimento-Analise-de->